

Strings, Numbers and Math

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

Length, Indexing, Immutability

```
let coffee = "espresso";  
coffee.length;    // 8  
coffee[0];        // "e"  
coffee.at(-1);    // "o" - negative index
```

- Strings are **immutable**: you cannot change a character in place

```
let word = "hello";  
word[0] = "H";    // silently ignored  
console.log(word); // "hello" (unchanged)
```

- Every string method returns a **new** string

Extracting Substrings

- `slice(start, end)`: end index is **exclusive**, supports negatives

```
let text = "Summit Coffee";  
text.slice(0, 6);    // "Summit"  
text.slice(7);       // "Coffee"  
text.slice(-6);      // "Coffee" (last 6 chars)
```

- `substring` is similar but ignores negatives; prefer `slice` in new code

Searching a String

```
let title = "The Summit Coffee Roasters";  
title.indexOf("Coffee");    // 11  
title.indexOf("Tea");       // -1 (not found)  
title.includes("Coffee");   // true  
title.startsWith("The");    // true  
title.endsWith("Roasters"); // true
```

- All are **case-sensitive**; lowercase both sides for insensitive search

```
title.toLowerCase().includes("coffee"); // true
```

Case, Trim, and Padding

```
"Latte".toUpperCase();    // "LATTE"  
"Latte".toLowerCase();    // "latte"  
  
"  alice@example.com  ".trim(); // "alice@example.com"  
  
"5".padStart(3, "0");      // "005"  
"7".padEnd(3, ".");        // "7.."  
`00:${String(9).padStart(2, "0")}`; // "00:09"
```

- `trim` is essential when handling form input

Split, Join, Replace, Repeat

```
"espresso,latte,mocha".split(",");  
// ["espresso", "latte", "mocha"]  
  
["a", "b", "c"].join(" | "); // "a | b | c"  
  
"a cat and a cat".replaceAll("cat", "dog");  
// "a dog and a dog"  
  
"=".repeat(10); // "=========="
```

- Prefer `replaceAll` over `replace` (which swaps only the first match)

Number Literals

- One `number` type for both integers and decimals

```
let count = 42;           // integer
let price = 4.5;          // decimal
let big = 1_000_000;       // _ separators (cosmetic)
let hex = 0xff;           // 255
let binary = 0b1010;      // 10
let scientific = 1.5e3;    // 1500
```

Parsing Text Into Numbers

- Form/URL/file input arrives as **strings**; convert it

```
Number("42");    // 42
Number("42px");   // NaN (strict: whole string must be valid)

Number.parseInt("42px", 10); // 42 (lenient: stops at "p")
Number.parseInt("3.9", 10);  // 3  (truncates)
Number.parseFloat("4.5rem"); // 4.5
```

- Always pass a **radix** (usually `10`) to `parseInt`

NaN: Not-a-Number

- Produced when a numeric operation fails
- Infamously **not equal to itself**

```
Number("hello"); // NaN  
NaN === NaN;      // false (!)
```

- So test with `Number.isNaN`, not `===`

```
let result = Number("hello");  
Number.isNaN(result); // true
```

- Prefer `Number.isNaN` over the misleading global `isNaN()`

Formatting Numbers

- `toFixed(n)` rounds to `n` decimals and returns a **string**

```
(4.5).toFixed(2);    // "4.50" (a string!)  
(3.14159).toFixed(2); // "3.14"
```

- Do not do math on the result; format only at display time
- `Intl.NumberFormat` for currency and separators

```
new Intl.NumberFormat("en-US", {  
  style: "currency", currency: "USD",  
}).format(1234.5); // "$1,234.50"
```

Floating Point and Radix

```
0.1 + 0.2;           // 0.30000000000000004  
0.1 + 0.2 === 0.3;  // false
```

- Not a JS bug; it is IEEE 754, the same in every language

```
(0.1 + 0.2).toFixed(2);           // "0.30" (round to display)  
Math.abs(0.1 + 0.2 - 0.3) < Number.EPSILON; // true (compare with tolerance)  
  
(255).toString(16); // "ff"  
(255).toString(2);  // "11111111"
```

- For money, store integer **cents** and divide by 100 to display

Rounding

- `Math` is a **static namespace**; never `new Math()`

```
Math.round(4.5); // 5    (nearest; .5 rounds up)
Math.floor(4.9); // 4    (down toward -Infinity)
Math.ceil(4.1);  // 5    (up toward +Infinity)
Math.trunc(4.9); // 4    (drop the decimal)
```

- `floor` vs `trunc` differ on **negatives**

```
Math.floor(-4.1); // -5  (down = more negative)
Math.trunc(-4.1); // -4  (chops, stays near zero)
```

Abs, Min, Max, Clamp

```
Math.abs(-7);           // 7 (distance from zero)
Math.max(3, 7, 2, 9, 1); // 9
Math.min(3, 7, 2, 9, 1); // 1

let temps = [61, 72, 58, 80];
Math.max(...temps); // 80 (spread an array)

// clamp volume into 0..100:
Math.min(100, Math.max(0, 130)); // 100
```

Powers and Roots

```
Math.pow(2, 10); // 1024
Math.sqrt(144);  // 12
Math.cbrt(27);   // 3
Math.hypot(3, 4); // 5 (straight-line distance)

2 ** 10;         // 1024 (** operator, often cleaner)
Math.sign(-5);   // -1
Math.PI;         // 3.141592653589793
```

Random Numbers

- `Math.random()` returns a float in `[0, 1)`, never quite 1
- The integer helper is the pattern to memorize (both ends inclusive)

```
function randomInt(min, max) {  
  return Math.floor(Math.random() * (max - min + 1)) + min;  
}  
  
randomInt(1, 6); // a dice roll: 1..6  
  
function pickRandom(array) {  
  return array[randomInt(0, array.length - 1)];  
}
```

- **Not secure**; use `crypto` APIs for tokens or passwords